

ASSIGNMENT - 4

Arrays — Declaring, Storing, Reading, Printing, Copying, Sorting, Searching, and Character Arrays (Strings) — IIITA Campus Life Problems

Course	Problem Solving with Programming (C)
Program	B.TECH — 2026 Batch
Section	IT-SEC A
Assignment No.	4
Subject Area	Arrays: declaration, memory storage, indexing, the address operator, initialisation, reading and printing, copying, reversing, sorting, searching, character arrays and strings, vowels, pattern matching
Theme	Real IIITA Campus Locations & Situations
Total Programs	10 (each with code, output and line-by-line explanation)

INSTRUCTIONS

Read the full PDF very carefully. Part A explains what an array is and every array concept of this assignment one by one, with a real IIITA example on the left and the matching C code on the right. Part B contains the 10 practice programs based on those concepts.

Today's Class Task

1. Run all **10 programs** given in Part B on your laptop or college system through VS Code.
2. Take a screenshot of the **code and output** for every question.
3. Paste all 10 screenshots into **one Google Doc**, in order from Question 1 to Question 10.
4. Name that Google Doc using your own **Enrollment Number**.
5. Submit it on **Google Classroom** by clicking the '**Turn In**' button.

Note — the comment lines inside the programs

Lines written as `// ...` or `/* ... */` inside a program are **comments**. The compiler ignores them completely, so they never change the output. They are written for the programmer: to keep the logic clear, and to remember why a line was written. Keep such comments in your own submitted programs as well.

Table of Contents

Part A — Concepts (with real IIITA examples)

1. What is an Array, and Why is it Needed? — the Letter Boxes at the Hostel
2. Declaring an Array — Reserving the Boxes
3. Accessing the Elements — the Index Starts at 0
4. Initialising an Array — Filling the Boxes at the Start
5. How an Array is Stored in Memory — One Long Row of Boxes
6. The Address Operator (&) — Why scanf Needs It
7. Reading Values into an Array with a Loop
8. Printing an Array — Forward and in Reverse
9. Copying One Array into Another — Step by Step
10. Working Through an Array — Largest, Sorting and Reversing
11. Character Arrays and Strings — the Hidden '\0'
12. Reading a String with %s, and Working Through a String
13. What an Array Gives You, and Where it Stops — advantages and limitations

Part B — Practice Programming Questions (10 Programs)

- Q1. Lab 5042 — Storing and Printing Five Marks (declare, initialise, print)
 - Q2. Main Gate — Reading Five Roll Numbers into an Array (scanf in a loop)
 - Q3. Badminton Court — Printing the Booked Slots in Reverse
 - Q4. IIITA Library — Copying One Shelf into Another, Step by Step
 - Q5. Lab 5042 — Finding the Highest Mark, for a Fixed Class and for n Students
 - Q6. Admin Office — Sorting the Marks in Ascending Order
 - Q7. IIITA Pavilion — Reversing the Seat Order Inside the Array
 - Q8. Auditorium Register — Reading a Name with %s and Counting the Letters
 - Q9. Extracting the Vowels from a Word
 - Q10. Pattern Matching — Searching for a Word Inside a Text
- Quick Revision** — Concept, Real IIITA Memory, Code Shape

PART A — CONCEPTS

Until now every variable you have used could hold **one** value at a time. `int marks;` holds one mark. If the class had five subjects, you needed five separate variables, and if it had sixty students you would need sixty names — which nobody can write, and no loop can go through.

An **array** solves exactly this. It gives one name to a **whole row of boxes**, and each box is picked by a number. That is the single idea behind this assignment; everything else is built on top of it.

Concept 1 explains it with something on the campus that you have seen many times, before any code appears.

What this assignment expects you to know already

Nothing about arrays is assumed — every part of it is built up from zero in Part A. What is reused are only the tools from your earlier assignments, and each one is named here so that you can look it up if it has slipped from memory.

TOOL	LEARNT IN	WHERE IT IS USED AGAIN HERE
<code>int</code> , <code>char</code> , <code>printf</code> , <code>scanf</code>	Assignment 1	everywhere
<code>%d</code> and <code>%c</code> format	Assignment 1 and 2	printing numbers and single letters
<code>if</code> and <code>else</code>	Assignment 2	Q5, Q6, Q9, Q10
<code>==</code> and <code>></code> comparisons	Assignment 2	finding the largest, sorting, matching letters
<code> </code> (logical OR)	Assignment 2	Q9, to test all the vowels in one condition
<code>for</code> and <code>while</code> loops	Assignment 3	every program of this assignment
nested loops	Assignment 3	Q6 (sorting) and Q10 (pattern matching)
<code>break</code>	Assignment 3	Q10, to stop as soon as the pattern is found

Everything else — the square brackets, the index, the address operator, the end mark of a string — is new, and is explained from the beginning in the concepts that follow.

CONCEPT 1

1. What is an Array, and Why is it Needed?

REAL IIITA SITUATION — THE LETTER BOXES AT THE HOSTEL

In the hostel entrance there is a **wooden board with a row of small boxes**, one for each room. The board has a single name written on it, and the boxes are simply numbered: 0, 1, 2, 3, 4.

Nobody says "the third box from the left, the brown one, next to the notice". Everyone says "**box number 3**" — one name for the whole board, plus a number for the box.

An **array** in C is exactly that board: **one name, many boxes, each box picked by its number**. All the boxes must hold the same kind of thing — a board of letters cannot suddenly hold a cricket bat.

WITHOUT AN ARRAY, AND WITH ONE

```
// five separate variables - hard work
int m1 = 78, m2 = 85, m3 = 62;
int m4 = 90, m5 = 71;
// no loop can visit m1 ... m5
```

```
// one array - one name, five boxes
int marks[5] = {78, 85, 62, 90, 71};

// and a loop CAN visit every box
for (i = 0; i < 5; i++)
    printf("%d ", marks[i]);
```

The same array drawn as the row of boxes

78 marks[0]	85 marks[1]	62 marks[2]	90 marks[3]	71 marks[4]
----------------	----------------	----------------	----------------	----------------

One name: marks · five boxes · each box picked by its number, and the numbering starts at **0**

AT THE LETTER-BOX BOARD	→	IN THE PROGRAM
The whole board	→	the array marks
One small box on the board	→	one element marks[2]
The number written under a box	→	the index 0, 1, 2, 3, 4
How many boxes the board has	→	the size int marks[5];
All boxes hold the same kind of thing	→	all elements have the same type int

Three words to fix in your mind

Array = the whole row of boxes, with one name.

Element = the value inside one box.

Index = the number that picks the box. In C the first index is **0**, not 1 — Concept 3 explains exactly what that means, and Concept 5 explains why C counts this way.

2. Declaring an Array

REAL IITB SITUATION — RESERVING THE BOXES

Before any letter arrives, the carpenter is told two things: **what kind of board** to make, and **how many boxes** it should have. Once the board is fixed to the wall, the number of boxes cannot change.

Declaring an array in C is the same order of information: first the **type** of value each box will hold, then the **name** of the board, then the **number of boxes** in square brackets.

The size must be a fixed number written by you. It cannot be decided later while the program is running.

DECLARING AN ARRAY IN C

```
int marks[5];
// type  name  [how many boxes]
```

PART	MEANING
int	every box holds a whole number
marks	the name of the whole row
[5]	five boxes are reserved
;	the declaration ends here

What the declaration alone produces

?	?	?	?	?
marks[0]	marks[1]	marks[2]	marks[3]	marks[4]

After `int marks[5];` the five boxes exist, but nobody has put anything in them yet. The question marks stand for whatever was left in that memory from before.

DECLARATION	WHAT IT RESERVES	VALID INDEX NUMBERS
<code>int marks[5];</code>	5 boxes, each holding an int	0, 1, 2, 3, 4
<code>int roll[60];</code>	60 boxes for 60 roll numbers	0 to 59
<code>float price[3];</code>	3 boxes, each holding a decimal number	0, 1, 2
<code>char name[20];</code>	20 boxes, each holding one character	0 to 19

AT THE CARPENTER	→	IN THE CODE
"Make boxes that hold whole numbers"	→	<code>int</code>
"Write this name on the board"	→	<code>marks</code>
"Make five of them"	→	<code>[5]</code>
"That is the whole order"	→	<code>;</code>
The board fixed to the wall for good	→	the size can never change later

Remember

Just after `int marks[5];` the five boxes exist but hold **rubbish values** — whatever was left in that memory from before. Printing them before putting anything in gives meaningless numbers. Concept 4 shows how to fill them at the start, and Concept 7 shows how to read them from the user.

3. Accessing the Elements — the Index Starts at 0

REAL IIITA SITUATION — COUNTING FROM THE CORNER

C does not count the boxes as "first, second, third". It counts **how far a box is from the start of the row**.

The very first box is at the start itself, so its distance is **0**. The next one is 1 box away, then 2, and so on.

So a board of 5 boxes is numbered **0, 1, 2, 3, 4** — and there is **no box number 5**. Asking for marks[5] is like reaching past the end of the board and grabbing whatever is nailed to the wall next to it.

READING AND CHANGING ONE ELEMENT

```
int marks[5] = {78, 85, 62, 90, 71};

printf("%d", marks[0]); // prints 78
printf("%d", marks[3]); // prints 90

marks[2] = 95; // box 2 now holds 95

// marks[5] does NOT exist
```

78

[0]

85

[1]

95

[2] changed

90

[3]

71

[4]

Green boxes were read, the orange box was changed by marks[2] = 95;

AT THE LETTER-BOX BOARD	→	IN THE CODE
The box at the very start of the row	→	marks[0]
The box three steps from the start	→	marks[3]
Putting a new slip into box 2	→	marks[2] = 95;
Reaching past the end of the board	→	marks[5] — it does not exist

NUMBER OF ELEMENTS	FIRST INDEX	LAST INDEX	THE LOOP THAT VISITS ALL OF THEM
5	0	4	for (i = 0; i < 5; i++)
6	0	5	for (i = 0; i < 6; i++)
n	0	n - 1	for (i = 0; i < n; i++)

The mistake almost everyone makes once

In an array loop the condition is $i < 5$, **not** $i \leq 5$. With $\leq$ the loop would also ask for marks[5], which does not exist. Inside a loop the compiler cannot see this coming, so it says nothing at all: the program builds, runs, and quietly prints a rubbish number in place of the sixth mark — or crashes. That silence is what makes the mistake dangerous. Whenever you write an array loop, count the boxes and use $i < \text{number of boxes}$.

4. Initialising an Array — Filling the Boxes at the Start

REAL IIITA SITUATION — THE BOARD IS FILLED BEFORE THE DAY BEGINS

On the first day of the session the hostel clerk puts a slip in every letter box before anyone arrives. The board and its contents are ready together.

That is **initialisation**: giving the array its values at the moment it is declared, using a list inside curly braces { }, with commas between the values.

If the clerk fills only the first two boxes, the remaining boxes are not left with rubbish — in C they are automatically set to **0**. And if he does not say how many boxes the board has but hands over five slips, the carpenter simply makes five boxes.

THE THREE WAYS TO INITIALISE

```
// 1. size given, all five values given
int marks[5] = {78, 85, 62, 90, 71};

// 2. size given, only some values given
int count[5] = {7, 3};
//    the rest become 0 automatically

// 3. size left empty, counted from list
int slot[] = {11, 12, 13};
//    C makes exactly 3 boxes
```

What each of the three lines really produces

78 [0]	85 [1]	62 [2]	90 [3]	71 [4]
-----------	-----------	-----------	-----------	-----------

`int marks[5] = {78, 85, 62, 90, 71};` — every box filled by hand

7 [0]	3 [1]	0 [2]	0 [3]	0 [4]
----------	----------	----------	----------	----------

`int count[5] = {7, 3};` — the three boxes not mentioned are set to 0 by C

11 [0]	12 [1]	13 [2]
-----------	-----------	-----------

`int slot[] = {11, 12, 13};` — the size is counted from the list, so exactly 3 boxes are made

AT THE HOSTEL BOARD	→	IN THE CODE
The clerk fills every box before the day starts	→	<code>int marks[5] = {78, 85, 62, 90, 71};</code>
He fills only the first two boxes	→	<code>int count[5] = {7, 3};</code> the rest become 0
He hands over five slips without saying how many boxes	→	<code>int slot[] = {11, 12, 13};</code> the size is counted
He tries to hand over more slips than boxes	→	<code>int a[3] = {1, 2, 3, 4};</code> the extra one is dropped

Two rules about the braces

1. The list may be shorter than the size, but it must **never be longer**. For `int a[3] = {1, 2, 3, 4};` the compiler reports *excess elements in array initializer* and quietly throws the fourth value away, so the program keeps only 1, 2 and 3.
2. The braces work only at the moment of declaration. Writing `marks = {78, 85};` on a later line is **not allowed** — after the declaration, values go in one box at a time.

5. How an Array is Stored in Memory

You can now declare an array, read a box and fill the boxes. This concept answers the question behind all of that: **why does the index start at 0, and why is the size fixed for ever?** The answer is in the way the boxes sit in memory.

REAL IIITA SITUATION — THE BOXES ARE SIDE BY SIDE, NOT SCATTERED

The letter boxes are not nailed to different walls of the hostel. They sit **side by side in one straight row**, and every box is the **same size**.

That is exactly how C keeps an array in memory: all the elements are stored **one after another, without gaps**, and each takes the same number of bytes.

An `int` usually takes **4 bytes**, so if the first box sits at address 1000, the next one starts at 1004, then 1008, and so on. This is why the computer can jump straight to any box: it knows where the row begins and how big each box is.

SEEING IT FOR YOURSELF

`sizeof` is a small built-in tool of C that answers one question: **how many bytes does this take in memory?** It is new in this assignment, and it is used here only to prove that the boxes really do sit side by side.

```
int marks[5] = {78, 85, 62, 90, 71};

printf("%d", (int) sizeof(marks));
// 20 : five boxes of 4 bytes each

printf("%d", (int) sizeof(marks[0]));
// 4 : one box
```

Total size of an array = **size of one element × number of elements**.

`sizeof` does not give a plain `int`, so `(int)` is written in front of it to print it with `%d`. Without that, the compiler gives a warning.

The five boxes as they really sit in memory

78 marks[0] 1000	85 marks[1] 1004	62 marks[2] 1008	90 marks[3] 1012	71 marks[4] 1016
------------------------	------------------------	------------------------	------------------------	------------------------

Addresses go up by 4 every time, because one `int` takes 4 bytes. The exact numbers differ on every machine; the **gap of 4** does not.

AT THE ROW OF BOXES	→	IN THE CODE
One box of the board	→	one <code>int</code> , 4 bytes
The whole board	→	$5 \times 4 = 20$ bytes
Where the row begins	→	the address of <code>marks[0]</code>
How C finds box <code>i</code>	→	start of the row + $i \times 4$

Why this matters

Because the boxes are in one unbroken row and all the same size, C can reach box number `i` by a single calculation: **start of the row + $i \times$ size of one box**. That is why `marks[500]` is found just as fast as `marks[0]`, and it is also why the size can never change after the declaration — the neighbouring memory already belongs to something else.

6. The Address Operator (&) — Why scanf Needs It

REAL IITA SITUATION — THE POSTMAN NEEDS THE BOX, NOT THE LETTER

When the postman comes to deliver a letter, telling him *what is written inside the letter* is useless. He needs to know **which box to put it in** — the location, not the contents.

The two things are different: `marks[2]` is the **value inside** box 2, while `&marks[2]` is the **address of** box 2.

`printf` only shows a value, so it needs no `&`. `scanf` has to **put** a value somewhere, so it must be given the address — exactly like the postman.

VALUE AGAINST ADDRESS

```
printf("%d", marks[2]); // the VALUE
scanf("%d", &marks[2]); // the ADDRESS
```

One exception worth knowing

The name of an array, written alone, already **is** the address of its first box. That is why a whole string is read with `scanf("%s", name);` and **no** `&` — Concept 12 comes back to this.

The same box, seen in the two ways

78 marks[0] 1000	85 marks[1] 1004	62 marks[2] 1008	90 marks[3] 1012	71 marks[4] 1016
------------------------	------------------------	------------------------	------------------------	------------------------

`marks[2]` is **62** — the value written inside the green box.

`&marks[2]` is **1008** — where that box stands. `printf` wants the first, `scanf` wants the second.

WRITTEN AS	MEANS	USED WITH
<code>marks[2]</code>	the value stored in box 2	<code>printf</code> , calculations, comparisons
<code>&marks[2]</code>	the address of box 2	<code>scanf</code> , when a value has to be put in
<code>marks</code>	the address of the first box, <code>&marks[0]</code>	reading a whole string with <code>%s</code>

AT THE LETTER-BOX BOARD	→	IN THE PROGRAM
Reading the slip that is inside box 2	→	<code>printf("%d", marks[2]);</code>
Telling the postman which box to fill	→	<code>scanf("%d", &marks[2]);</code>
Pointing at where the whole board starts	→	<code>marks</code>

7. Reading Values into an Array with a Loop

REAL IIITA SITUATION — FILLING THE BOXES ONE BY ONE

The clerk does not fill the whole board in one movement. He stands at box 0, puts a slip in, moves to box 1, puts a slip in, and continues to the last box.

A program does the same: one `scanf` inside a loop, with the loop counter used as the index. The counter does two jobs at once — it moves the clerk along, and it picks the box.

Notice that the same counter `i` appears in two places: `&roll[i]` chooses the box, and `i++` moves to the next one.

READING FIVE VALUES

```
int roll[5];
int i;

for (i = 0; i < 5; i++) {
    scanf("%d", &roll[i]);
}
```

The user types the five numbers separated by spaces or by pressing Enter; `scanf` takes them one at a time.

Step by step, as the five numbers 11, 12, 13, 14, 15 are typed

<code>i = 0</code>	→	<code>scanf</code> puts 11 into <code>roll[0]</code>
<code>i = 1</code>	→	<code>scanf</code> puts 12 into <code>roll[1]</code>
<code>i = 2</code>	→	<code>scanf</code> puts 13 into <code>roll[2]</code>
<code>i = 3</code>	→	<code>scanf</code> puts 14 into <code>roll[3]</code>
<code>i = 4</code>	→	<code>scanf</code> puts 15 into <code>roll[4]</code> — the last box
<code>i = 5</code>	→	<code>5 < 5</code> is false, so the loop stops. Box 5 is never touched, and that is correct.

AT THE HOSTEL BOARD	→	IN THE CODE
The clerk standing in front of box <code>i</code>	→	<code>roll[i]</code>
Telling him which box to fill	→	<code>&roll[i]</code>
Dropping one slip in	→	<code>scanf("%d", &roll[i]);</code>
Stepping to the next box	→	<code>i++</code>

11 roll[0]	12 roll[1]	13 roll[2]	14 roll[3]	15 roll[4]
---------------	---------------	---------------	---------------	---------------

The board after the loop has finished

Three things that go wrong while reading

- 1. The & is left out.** `scanf("%d", roll[i]);` passes the value in the box instead of its address. The program may print rubbish or stop suddenly, and the compiler usually gives a warning — read that warning, it is pointing at this line.
- 2. The loop goes one step too far.** `i <= 5` would ask for `roll[5]`, which does not exist.
- 3. Fewer numbers are typed than the loop asks for.** The program simply waits, because `scanf` has not been given its fifth value yet. Type the missing number and press Enter, and it will continue.

8. Printing an Array — Forward and in Reverse

REAL IITB SITUATION — WALKING ALONG THE BOARD, AND WALKING BACK

To read the whole board you walk from the first box to the last: 0, 1, 2, 3, 4.

To read it backwards you simply **start at the far end and walk towards the corner**: 4, 3, 2, 1, 0. The board is not rearranged at all — only the direction of your walk changes.

In C, that is the difference between counting the index **up** and counting it **down**. Nothing inside the array moves.

THE TWO LOOPS

```
// forward: 0 to 4
for (i = 0; i < 5; i++) {
    printf("%d ", slot[i]);
}
```

```
// reverse: 4 down to 0
for (i = 4; i >= 0; i--) {
    printf("%d ", slot[i]);
}
```

The same row, read in two directions



Forward: start at the green box and walk right, 0 → 5. Reverse: start at the orange box and walk left, 5 → 0.
Either way the values inside the boxes are never moved — only the order of reading changes.

WALKING ALONG THE BOARD	→	IN THE CODE
Starting at the first box	→	<code>i = 0</code>
Starting at the last box	→	<code>i = n - 1</code>
Walking towards the end	→	<code>i++</code>
Walking back towards the start	→	<code>i--</code>
Reading the slip in the box you stand at	→	<code>printf("%d ", slot[i]);</code>

PART OF THE LOOP	FORWARD	REVERSE	WHY
Start	<code>i = 0</code>	<code>i = 4</code>	begin at the first box, or at the last one
Condition	<code>i < 5</code>	<code>i >= 0</code>	index 0 must still be printed, so <code>>=</code> is needed
Update	<code>i++</code>	<code>i--</code>	walk towards the end, or back towards the corner

Two easy mistakes in the reverse loop

- Starting at `i = 5` instead of `i = 4` — box 5 does not exist. For an array of `n` boxes the reverse loop starts at `n - 1`.
- Writing `i > 0` instead of `i >= 0` — the loop would stop before printing the very first box.

9. Copying One Array into Another — Step by Step

REAL IIITA SITUATION — MOVING THE BOOKS TO A SECOND SHELF

The Library has a shelf of five books, and the librarian wants the same five books arranged on a second, empty shelf.

He cannot pick up the shelf itself and duplicate it. He takes the **first book, places it in the first slot of the new shelf**, then the second book into the second slot, and so on — five separate moves.

C works the same way. Writing `shelfB = shelfA;` is **not allowed**, because the name of an array is the fixed address of its row of boxes and cannot be reassigned. The copy has to be made **one element at a time**, inside a loop.

THE WRONG WAY AND THE RIGHT WAY

```
// WRONG - this will not compile
shelfB = shelfA;
```

```
// RIGHT - one box at a time
for (i = 0; i < 5; i++) {
    shelfB[i] = shelfA[i];
}
```

Read the assignment from right to left: **take what is in shelfA box i, and put it into shelfB box i.**

The five moves, one by one

21 shelfA[0]	34 shelfA[1]	17 shelfA[2]	48 shelfA[3]	56 shelfA[4]
-----------------	-----------------	-----------------	-----------------	-----------------

shelfA — the shelf being copied from. It is never changed.

i = 0	→	shelfB[0] = shelfA[0] → 21 is now in the new shelf
i = 1	→	shelfB[1] = shelfA[1] → 34 copied
i = 2	→	shelfB[2] = shelfA[2] → 17 copied
i = 3	→	shelfB[3] = shelfA[3] → 48 copied
i = 4	→	shelfB[4] = shelfA[4] → 56 copied, the shelf is complete

21 shelfB[0]	34 shelfB[1]	17 shelfB[2]	48 shelfB[3]	56 shelfB[4]
-----------------	-----------------	-----------------	-----------------	-----------------

shelfB after the loop — the same five values, but in its own boxes. Changing shelfB later does not touch shelfA.

AT THE LIBRARY SHELVES	→	IN THE CODE
The book standing in slot i of shelf A	→	<code>shelfA[i]</code>
The empty slot i of shelf B	→	<code>shelfB[i]</code>
Carrying that one book across	→	<code>shelfB[i] = shelfA[i];</code>
Doing it for every slot in turn	→	<code>for (i = 0; i < 5; i++)</code>

Two rules for copying

1. One name cannot be given to another. `shelfB = shelfA;` is refused by the compiler, because the name of an array is the fixed address of its row of boxes. Only the values can be moved, and only one box at a time.

2. The second array must be big enough. Copying five values into an array declared as `int shelfB[3];` would write past the end of the row — the same out-of-range mistake as `marks[5]` in Concept 3, and just as dangerous.

10. Working Through an Array — Largest, Sorting and Reversing

Once values are inside an array, almost every useful task is the same pattern: **walk through the boxes with a loop and do something at each step**. Three of those tasks appear in Questions 5, 6 and 7, and all three are built on that one idea.

No new loop is needed for any of them. The for and while loops, the **nested loops** and break are all from Assignment 3, and the `||` used for the vowels is from Assignment 2. The only new part is the square brackets that pick a box.

Finding the largest of n numbers

REAL SITUATION — THE HIGHEST MARK IN THE CLASS

You look at the first paper and keep it in your hand as "the best so far". Then you compare every next paper with the one in your hand, and swap only when a better one appears. At the end, the paper in your hand is the highest.

IN C

```
int largest = marks[0]; // best so far

for (i = 1; i < 6; i++) {
    if (marks[i] > largest) {
        largest = marks[i];
    }
}
```

The loop starts at `i = 1`, because box 0 is already in your hand.

The walk, seen as boxes

62 [0] in hand	91 [1] beats it	45 [2]	78 [3]	88 [4]	53 [5]
-------------------	--------------------	-----------	-----------	-----------	-----------

Only one box ever beats the value in your hand, so `largest` changes once, at box 1, and then stays 91.

WITH THE ANSWER SHEETS	→	IN THE CODE
The sheet you keep in your hand	→	<code>largest</code>
Comparing the next sheet with it	→	<code>if (marks[i] > largest)</code>
Swapping two neighbouring sheets	→	<code>temp = marks[j]; marks[j] = marks[j+1]; marks[j+1] = temp;</code>
Turning the whole row around	→	<code>swap marks[i] with marks[n - 1 - i]</code>

Sorting the values into order

REAL SITUATION — ARRANGING ANSWER SHEETS BY MARKS

You compare two neighbouring sheets, and if the left one is bigger you swap them. Going along the row again and again, the biggest sheet keeps moving to the end, then the next biggest, until everything is in order.

IN C — A NESTED LOOP (ASSIGNMENT 3) AND A SWAP

```
for (i = 0; i < 5; i++) {  
    for (j = 0; j < 5 - i; j++) {  
        if (marks[j] > marks[j + 1]) {  
            temp = marks[j];  
            marks[j] = marks[j + 1];  
            marks[j + 1] = temp;  
        }  
    }  
}
```

Why a swap needs a third box

Two values cannot be exchanged directly. Writing `a = b; b = a;` loses the old value of `a` for ever. A spare variable is needed to hold it, exactly like needing an empty hand to swap two books: `temp = a; a = b; b = temp;`

Reversing an array — printing it backwards is not the same thing

TASK	THE IDEA IN ONE LINE	WHERE IT APPEARS
Printing in reverse	walk the index from <code>n - 1</code> down to 0; the array itself is not changed	Question 3
Reversing the array itself	swap the first with the last, the second with the second-last, and stop at the middle	Question 7

The pattern behind all three

Take a counter, walk it through the boxes, and at every box either **compare**, **print**, or **move** something. Nothing more complicated than that is happening in Questions 5, 6 and 7. The same pattern is used again for strings in Concept 12.

11. Character Arrays and Strings — the Hidden '\0'

REAL IIITA SITUATION — WRITING A NAME ON THE NOTICE BOARD

A name written on the notice board is not one single thing to the computer. It is a **row of letters**, one letter in each box: S, u, b, r, a, t, a.

So a **string in C is simply an array of characters**. There is no separate "string" type.

But how does the computer know where the name ends, if the board has 20 boxes and the name uses only 5? C puts a **special end mark** in the next box: the character '\0', called the null character. Every function that prints or measures a string reads letters until it meets that mark and then stops.

This is why `char name[20];` can hold a name of at most **19 letters** — the last box is needed for the end mark.

ONE CHARACTER AGAINST A STRING

```
char grade = 'A';
// single quotes: ONE character

char name[20] = "Subrata";
// double quotes: a whole string,
// and C adds '\0' at the end
```

```
// counting the letters yourself
i = 0;
while (name[i] != '\0') {
    i++;
}
// i is now 7
```

The name Subrata inside a char array of 20 boxes

S [0]	u [1]	b [2]	r [3]	a [4]	t [5]	a [6]	\0 [7] end mark	? [8]	? [9]
----------	----------	----------	----------	----------	----------	----------	--------------------	----------	----------

Seven letters, then the end mark in box 7. Boxes 8 to 19 exist but are never read, because printing stops at '\0'.

ON THE NOTICE BOARD	→	IN THE CODE
One single letter	→	<code>char grade = 'A';</code> single quotes
A whole name	→	<code>char name[20] = "Subrata";</code> double quotes
The mark that says the name ends here	→	<code>'\0'</code> , added by C
Counting the letters yourself	→	<code>while (name[i] != '\0') i++;</code>

Printing a character array: %c for one box, %s for the whole row

FORMAT	PRINTS	EXAMPLE	RESULT FOR NAME = "SUBRATA"
%c	the single character in one box	<code>printf("%c", name[0]);</code>	S
%s	the whole row, from box 0 up to the end mark	<code>printf("%s", name);</code>	Subrata

You already used %c in Assignment 2 for a single char variable. It works in exactly the same way here; the only difference is that the character now comes out of one box of an array.

Remember

'A' with single quotes is one character and fits in a char. "A" with double quotes is a string of one letter plus the end mark, so it needs **two** boxes. This is why a char array must always be declared a little larger than the longest word you expect.

12. Reading a String with %s, and Working Through a String

REAL IIITA SITUATION — WRITING YOUR NAME IN THE REGISTER

At the Auditorium register you write your whole name in one go, not letter by letter. C can do the same with the **%s** format: it reads a run of characters and drops them into the array together, then adds the `'\0'` end mark itself.

Two things about %s often surprise beginners:

- It takes **no** &, because the array name already is the address of the first box.
- It stops at the **first space**. Typing "Subrata Pramanik" stores only "Subrata".

READING AND PRINTING A NAME

```
char name[20];

printf("Enter your name: ");
scanf("%s", name);      // no & here

printf("%s", name);     // prints it back
```

READING	WRITTEN AS
one int	<code>scanf("%d", &n);</code>
one element of an int array	<code>scanf("%d", &roll[i]);</code>
a whole string	<code>scanf("%s", name);</code>

Why %s needs no &

For a single variable, &n has to be written because n by itself means the *value*. But an array name by itself already means the **address of the first box**, so name is exactly what scanf wants. Writing &name is a common mistake — it happens to work on many compilers, but it is the wrong idea, and in this course you should write `scanf("%s", name);`.

Working through a string — the same walk, stopped by the end mark

A number array is walked with a counter that stops at a known size, as in Concept 10. A string is walked the same way, but the stopping rule is different: there is no size to count to, so the loop stops at the **end mark** `'\0'` instead.

WALKING A NUMBER ARRAY

```
for (i = 0; i < 6; i++) {
    // stop after 6 boxes
}
```

WALKING A STRING

```
while (word[i] != '\0') {
    i++; // stop at the end mark
}
```

Where the walk stops, seen as boxes



The condition `word[i] != '\0'` is true for boxes 0 to 6 and false at box 7. Boxes 8 onwards are never looked at, so their rubbish values do no harm.

WHILE READING A WORD	→	IN THE CODE
Writing the whole name at once	→	<code>scanf("%s", name);</code>
Looking at one letter	→	<code>word[i]</code>
Knowing the word has ended	→	<code>word[i] == '\0'</code>
Moving to the next letter	→	<code>i++</code>

TASK	THE IDEA IN ONE LINE	WHERE IT APPEARS
Extracting the vowels	walk to the end mark, and print the letter only if it is a, e, i, o or u — using <code> </code> from Assignment 2	Question 9
Pattern matching	at every position of the text, compare the pattern letter by letter; if all of them match, the pattern is found there	Question 10

What is new here, and what is not

The loops, break and `||` are all from Assignments 2 and 3. The only new ideas in Questions 9 and 10 are the ones you have just learnt: a string is an array of characters, and the walk stops at `'\0'`.

13. What an Array Gives You, and Where it Stops

Every tool in programming is chosen for what it does well, and every tool has a point where it stops being the right choice. Now that all twelve concepts are done, this page puts both sides of the array in one place — **why it is used**, and **what it cannot do**. Each row also says where in this assignment you have already met that behaviour.

What an array gives you

ADVANTAGE	WHAT IT MEANS IN PRACTICE	SEEN IN
One name for many values	Sixty marks need one name, marks, not sixty variable names that nobody could write or remember.	Concept 1
A loop can visit every value	The index is a number, so it can be counted by a loop. Five separate variables cannot be reached by any loop at all.	Concept 1, every question
Any box is reached instantly	The boxes sit side by side and are all the same size, so C finds box 500 in one calculation, as fast as box 0.	Concept 5
Reading many values is one small loop	Whether it is 5 numbers or 500, the reading code is the same three lines.	Concept 7, Q2
Text becomes possible	A word is just an array of characters with an end mark, so names and sentences can be stored and searched.	Concepts 11 and 12

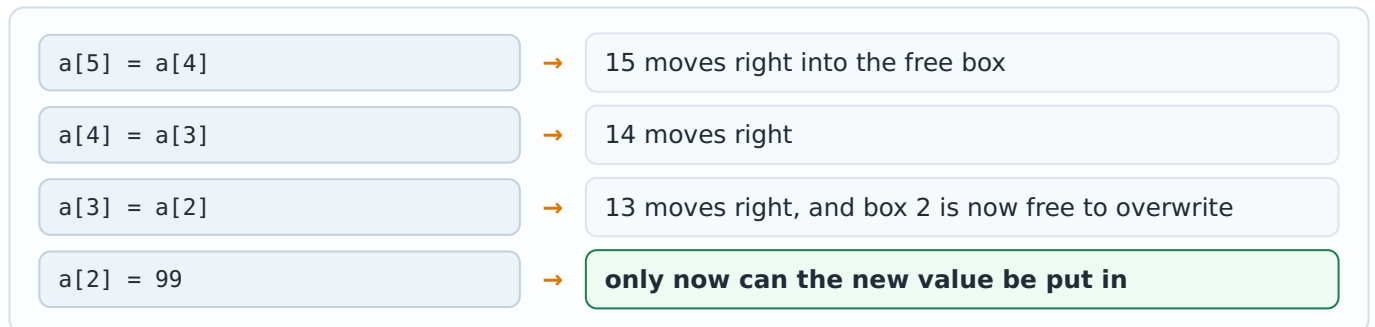
Where an array stops

LIMITATION	WHAT HAPPENS, AND WHAT TO DO ABOUT IT	SEEN IN
The size is fixed for ever	The number in the brackets must be a fixed number written by you, and it cannot grow later. The usual answer is to declare the array comfortably large and use only the first n boxes.	Concept 2, Q5
C does not check the boundary	Asking for marks[5] in an array of five is not stopped inside a loop. The program runs and prints a rubbish value, or crashes. The programmer is the only guard.	Concept 3
One array cannot be copied with =	shelfB = shelfA; is refused by the compiler. Copying has to be done one box at a time in a loop.	Concept 9, Q4
Two arrays cannot be compared with ==	if (a == b) does not compare the values. It compares where the two rows sit in memory, so it is false even when every value is equal. gcc warns: <i>comparison between two arrays</i> . Values must be compared box by box in a loop.	the same idea as Concept 9
%s stops at the first space	Typing "Subrata Pramanik" stores only "Subrata". A full line needs a different tool, which comes in a later topic.	Concept 12, Q8
Inserting in the middle is expensive	To put a value into box 2, every value from box 2 onwards must first be shifted one place to the right. For a long array that is a lot of moving.	the swap idea of Concept 10

The last one, seen as boxes



Before: a new value, 99, has to go into box 2



After: three values had to be moved (green) before one value could be inserted (orange)

WHAT YOU WANT TO DO	→	IN THE CODE
Keep many values of one type	→	use an array
Visit every value with a loop	→	use an array – the index is a number
Grow the collection while the program runs	→	an array cannot; the size is fixed
Copy or compare two whole collections	→	not with = or ==; a loop, box by box
Insert into the middle again and again	→	expensive – everything after it must shift

How to decide, in one sentence

Use an array when you have **many values of the same type** and you know roughly **how many** there can be, and when you mostly want to **read them, count them or run through them**. Look for something else when the number cannot be guessed at all, or when values must be constantly inserted and removed from the middle.

PART B — PRACTICE PROGRAMMING QUESTIONS

Each of the 10 programs below is built on a concept from Part A, and is given with its real IIITA situation, the complete program, a line-by-line explanation, and the exact output.

Which concept each question uses

QUESTION	CONCEPT USED
Q1 — Five marks of a student	declaring, initialising and printing an array (Concepts 2, 4 and 8)
Q2 — Reading five roll numbers	scanf in a loop and the address operator (Concepts 6 and 7)
Q3 — Booked slots in reverse	printing an array backwards (Concept 8)
Q4 — Copying one shelf into another	element-by-element copying (Concept 9)
Q5 — Highest mark, fixed class and n students	walking through an array to find the largest, and using a size read from the user (Concepts 7 and 10)
Q6 — Sorting the marks	nested loops and swapping (Concept 10)
Q7 — Reversing the seat order	swapping the two ends of an array (Concept 10)
Q8 — Reading a name with %s	character arrays, '\0' and the %s format (Concepts 11 and 12)
Q9 — Vowels in a word	walking a string to the end mark (Concepts 11 and 12)
Q10 — Searching a word inside a text	pattern matching with nested loops (Concept 12)

Q1. Lab 5042 — Storing and Printing Five Marks

REAL SITUATION

Result day — the marks of one student in Lab 5042.

The student has marks in **five subjects**: 78, 85, 62, 90 and 71.

Instead of five separate variables, all five are kept in **one array of five boxes**, and a loop reads out every box with its index.

CONCEPT USED

Declaration with a size, **initialisation** inside braces, and **printing** with a for loop.

The loop condition is $i < 5$, so the index runs 0, 1, 2, 3, 4 — the five boxes that exist.

```
#include <stdio.h>

int main() {
    int marks[5] = {78, 85, 62, 90, 71}; // five boxes, filled at the start
    int i;

    printf("Marks stored in the array:\n");

    for (i = 0; i < 5; i++) {           // index 0 to 4
        printf("marks[%d] = %d\n", i, marks[i]);
    }

    return 0;
}
```

78

marks[0]

85

marks[1]

62

marks[2]

90

marks[3]

71

marks[4]

The array as the program creates it

IN THE MARK SHEET	→	IN THE CODE
Five subject marks of one student	→	int marks[5] = {78, 85, 62, 90, 71};
The mark of the very first subject	→	marks[0]
Reading out every mark in turn	→	for (i = 0; i < 5; i++)
Showing which box each mark came from	→	printf("marks[%d] = %d", i, marks[i]);

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int marks[5] = { ... };</code>	Reserves five boxes and fills them in the same line. The number of values inside the braces must not be more than 5.
<code>int i;</code>	The counter that will pick the boxes. It is a normal int, not part of the array.
<code>for (i = 0; i < 5; i+ +)</code>	Starts at 0 because the first box is marks[0], and stops before 5 because marks[5] does not exist.
<code>printf(..., i, marks[i]);</code>	Prints the index and the value in that box. Two %d are used, so two values are given in the same order.
<code>printf("Marks stored...\n");</code>	A heading line, printed once before the loop so that the five lines below it have a title. The \n pushes the first mark onto its own line.

OUTPUT

```
Marks stored in the array:
marks[0] = 78
marks[1] = 85
marks[2] = 62
marks[3] = 90
marks[4] = 71
```

Q2. Main Gate — Reading Five Roll Numbers into an Array

REAL SITUATION

Morning — the guard at the IIITA Main Gate.

He has to note the roll numbers of the **first five students** who enter. He writes them into the register one at a time, in the order they arrive.

The program does the same: five boxes are reserved first, then a loop puts one typed number into each box.

CONCEPT USED

scanf inside a loop, with the **address operator**.

`&roll[i]` is written because `scanf` has to know **which box to fill**, not what is already in it.

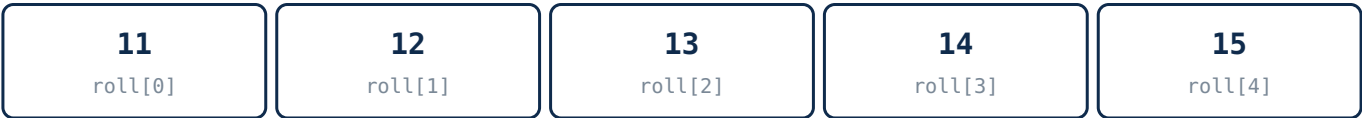
```
#include <stdio.h>

int main() {
    int roll[5];           // five empty boxes
    int i;

    printf("Enter 5 roll numbers: ");
    for (i = 0; i < 5; i++) {
        scanf("%d", &roll[i]);    // & gives the address of box i
    }

    printf("Roll numbers stored in the array:\n");
    for (i = 0; i < 5; i++) {
        printf("roll[%d] = %d\n", i, roll[i]);
    }

    return 0;
}
```



The array after the loop has read all five numbers

AT THE MAIN GATE REGISTER	→	IN THE CODE
Room for five roll numbers	→	<code>int roll[5];</code>
The guard writing down one roll number	→	<code>scanf("%d", &roll[i]);</code>
Moving to the next line of the register	→	<code>i++</code>
Reading the register back afterwards	→	the second for loop

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int roll[5];</code>	Only reserves the boxes. Until the loop runs they hold rubbish values, so nothing is printed before reading.
<code>scanf("%d", &roll[i]);</code>	Puts the typed number into box i. Without the & the program would pass a value instead of a place, and would not work.
the first loop	Fills the array. The five numbers can be typed with spaces between them, or with Enter after each one.
the second loop	Reads the array back. Reading and printing are kept in two separate loops so that each loop does one job.
<code>int i;</code>	The counter used by both loops. One counter is enough, because the first loop has finished before the second one starts.
<code>printf("Enter 5 roll numbers: ");</code>	The prompt. It is printed once, before the reading loop, so the user knows what to type.
<code>printf("roll[%d] = %d\n", ...);</code>	Prints the index and the value together, so it is visible which box each number went into.

OUTPUT (THE FIVE NUMBERS 11 12 13 14 15 ARE TYPED)

```
Enter 5 roll numbers: 11 12 13 14 15
Roll numbers stored in the array:
roll[0] = 11
roll[1] = 12
roll[2] = 13
roll[3] = 14
roll[4] = 15
```

Q3. Badminton Court — Printing the Booked Slots in Reverse

REAL SITUATION

The booking chart of the badminton court.

Six slots have been booked, in the order 11, 12, 13, 14, 15, 16.

The court secretary first reads the list from the top, and then reads it again **from the bottom upwards** to announce the last booking first. The chart itself is not rewritten — only the direction of reading changes.

CONCEPT USED

Printing an array forward and in reverse.

Forward: $i = 0 \dots i < 6 \dots i++$

Reverse: $i = 5 \dots i \geq 0 \dots i--$

The reverse loop starts at **5**, not 6, and the condition is $\geq$ so that box 0 is also printed.

```
#include <stdio.h>

int main() {
    int slot[6] = {11, 12, 13, 14, 15, 16};
    int i;

    printf("Booked slots in order:\n");
    for (i = 0; i < 6; i++) {           // six boxes, so the index stops at 5
        printf("%d ", slot[i]);
    }

    printf("\nSame slots in reverse order:\n");
    for (i = 5; i >= 0; i--) {         // 5 down to 0
        printf("%d ", slot[i]);
    }

    return 0;
}
```

The same six boxes, read twice



The first loop walks green → orange, the second walks orange → green. The values inside the boxes are the same both times.

Step by step — the second loop walking backwards

<code>i = 5</code>	→	prints <code>slot[5] = 16</code> , then <code>i--</code> makes <code>i = 4</code>
<code>i = 4, 3, 2, 1</code>	→	prints 15, 14, 13 and 12 in that order
<code>i = 0</code>	→	0 >= 0 is still true, so <code>slot[0] = 11</code> is printed as well
<code>i = -1</code>	→	<code>-1 >= 0</code> is false, so the loop ends. With <code>i > 0</code> the first slot would have been missed.

ON THE BOOKING CHART	→	IN THE CODE
Six slots already booked	→	<code>int slot[6] = {11, ... , 16};</code>
Reading the chart from the top	→	<code>for (i = 0; i < 6; i++)</code>
Reading the same chart from the bottom	→	<code>for (i = 5; i >= 0; i--)</code>
The chart itself is never rewritten	→	the array is not changed

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>for (i = 0; i < 6; i++)</code>	The forward walk. Six boxes, so the index goes 0 to 5.
<code>for (i = 5; i >= 0; i--)</code>	The backward walk. It starts at <code>6 - 1 = 5</code> , and <code>i--</code> moves towards the front.
<code>i >= 0</code>	<code>>=</code> is needed, not <code>></code> . With <code>i > 0</code> the loop would stop before printing <code>slot[0]</code> and the first booking would be lost.
the array is unchanged	Nothing was moved inside the array. Reversing the printing is not the same as reversing the array — that is Question 7.
<code>int slot[6] = { ... };</code>	Six boxes filled at the start. The values are the booked slot numbers.
<code>int i;</code>	The counter. The same variable is used for both directions; only its starting value and its update change.
the two heading <code>printf</code> lines	Printed once each, outside the loops, to label the two lists. The <code>\n</code> at the start of the second one moves it to a fresh line.
<code>printf("%d ", slot[i]);</code>	Prints one value with a space after it, so all six appear on one line instead of six separate lines.

OUTPUT

```
Booked slots in order:
11 12 13 14 15 16
Same slots in reverse order:
16 15 14 13 12 11
```

Q4. IITA Library — Copying One Shelf into Another

REAL SITUATION

The Library is rearranging its shelves.

Shelf A holds five books with the code numbers 21, 34, 17, 48 and 56. The same five codes have to appear on the empty shelf B, in the same order.

The librarian moves them **one book at a time**, and the program prints each move as it happens, so every step can be seen.

CONCEPT USED

Copying element by element.

shelfB = shelfA; is not allowed in C. The copy must be made inside a loop, one box at a time:

```
shelfB[i] = shelfA[i];
```

```
#include <stdio.h>

int main() {
    int shelfA[5] = {21, 34, 17, 48, 56};
    int shelfB[5];           // the empty shelf
    int i;

    for (i = 0; i < 5; i++) {
        shelfB[i] = shelfA[i];    // one book moved
        printf("Step %d: shelfA[%d] = %d copied into shelfB[%d]\n", i + 1, i, shelfA[i], i);
    }

    printf("\nshelfB after copying: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", shelfB[i]);
    }

    return 0;
}
```



AT THE LIBRARY SHELVES	→	IN THE CODE
The shelf holding five books	→	int shelfA[5] = {21, ... , 56};
The empty second shelf	→	int shelfB[5];
Moving one book across	→	shelfB[i] = shelfA[i];
Announcing each move out loud	→	printf("Step %d ...", i + 1, ...);

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int shelfB[5];</code>	Reserves the second shelf. It must have at least as many boxes as the first one.
<code>shelfB[i] = shelfA[i];</code>	The copy itself. Read it from right to left: take the value in shelfA box i and put it into shelfB box i.
<code>i + 1</code> in the message	Only for the reader. The boxes are numbered from 0, but people count steps from 1, so the step number is printed as <code>i + 1</code> .
the second loop	Prints shelfB after the work is done, to show that the copy really happened. shelfA is never changed.
<code>int shelfA[5] = { ... };</code>	The first shelf, filled at the start. It is only read during the copy, never changed.
<code>int i;</code>	The counter that picks the same box number in both arrays.
<code>printf("\nshelfB after copying: ");</code>	A label printed once after the loop. The leading <code>\n</code> separates it from the five step messages.

OUTPUT

```
Step 1: shelfA[0] = 21 copied into shelfB[0]
Step 2: shelfA[1] = 34 copied into shelfB[1]
Step 3: shelfA[2] = 17 copied into shelfB[2]
Step 4: shelfA[3] = 48 copied into shelfB[3]
Step 5: shelfA[4] = 56 copied into shelfB[4]

shelfB after copying: 21 34 17 48 56
```

Q5. Lab 5042 — Finding the Highest Mark in the Class

REAL SITUATION

Six answer sheets from the Lab 5042 test:
62, 91, 45, 78, 88 and 53.

You pick up the first sheet and keep it in your hand as "the best so far". Then you look at every other sheet: if it is better than the one in your hand, it becomes the new best.

After the last sheet, the one in your hand is the highest mark of the class.

CONCEPT USED

Walking through an array while keeping one value.

largest starts as marks[0] — the sheet already in your hand — so the loop begins at i = 1.

```
#include <stdio.h>

int main() {
    int marks[6] = {62, 91, 45, 78, 88, 53};
    int i;
    int largest = marks[0];           // the best so far

    for (i = 1; i < 6; i++) {         // start at 1, box 0 is already in hand
        if (marks[i] > largest) {
            largest = marks[i];       // a better one is found
        }
    }

    printf("Marks: ");
    for (i = 0; i < 6; i++) {
        printf("%d ", marks[i]);
    }
    printf("\nHighest mark in the class = %d", largest);

    return 0;
}
```

The six marks, and the one the loop keeps

62 [0] first in hand	91 [1] the winner	45 [2]	78 [3]	88 [4]	53 [5]
----------------------------	----------------------	-----------	-----------	-----------	-----------

The loop starts holding box 0. Only box 1 ever beats it, so largest becomes 91 and stays 91 for the rest of the walk.

Step by step

start	→	largest = marks[0] = 62
i = 1	→	91 > 62 → largest becomes 91
i = 2	→	45 > 91 is false → largest stays 91
i = 3	→	78 > 91 is false → largest stays 91
i = 4	→	88 > 91 is false → largest stays 91
i = 5	→	53 > 91 is false → largest stays 91
end	→	the loop is over, the highest mark is 91

WITH THE ANSWER SHEETS	→	IN THE CODE
The sheet kept in your hand	→	<code>int largest = marks[0];</code>
Picking up the next sheet	→	<code>for (i = 1; i < 6; i++)</code>
"Is this one better than mine?"	→	<code>if (marks[i] > largest)</code>
Keeping the better sheet instead	→	<code>largest = marks[i];</code>

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int largest = marks[0];</code>	The first value is taken as the best so far. Starting from 0 instead would be wrong if all the marks were negative.
<code>for (i = 1; ...)</code>	Starts at 1, because box 0 has already been used as the starting value. Starting at 0 would only compare 62 with itself once, which changes nothing.
<code>if (marks[i] > largest)</code>	The comparison. Change > to < and the same program finds the lowest mark instead.
<code>int marks[6] = { ... };</code>	The six marks of the class, filled at the start so that no input is needed.
<code>int i;</code>	The counter for both the search loop and the printing loop.
<code>printf("Marks: ");</code> and the loop	Prints all six marks on one line first, so that the answer below can be checked by eye.
<code>printf("\nHighest mark ... %d", largest);</code>	Prints the result once, after the search is complete. Printing it inside the loop would show every intermediate best value instead.

OUTPUT

```
Marks: 62 91 45 78 88 53
Highest mark in the class = 91
```

The same job when the number of students is not fixed

In the program above the class size was six, written into the code. In a real office the size changes every time, so the program should **ask how many marks there are** and then read exactly that many. Only two things change: the array is declared large enough for the biggest class expected, and every loop now runs to n instead of to 6.

WHAT IS DIFFERENT

- `int marks[50];` — room for up to 50 students, but only the first n boxes are used
- n is read first, before any mark
- every loop condition becomes $i < n$
- the boxes from n to 49 are simply left alone

WHY THE ARRAY IS STILL DECLARED WITH 50

The size inside the square brackets must be a fixed number written by you (Concept 2), so it cannot be `marks[n]`. The usual way is to declare it comfortably large and use only as much as needed.

If a class of 60 came along, the program would need `marks[60]` or more — asking for `marks[55]` in an array of 50 is the out-of-range mistake of Concept 3.

```
#include <stdio.h>

int main() {
    int marks[50];           // room for up to 50 students
    int n, i;
    int largest;

    printf("How many students are there? ");
    scanf("%d", &n);

    printf("Enter %d marks: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &marks[i]);    // only the first n boxes are used
    }

    largest = marks[0];        // the best so far
    for (i = 1; i < n; i++) {
        if (marks[i] > largest) {
            largest = marks[i];
        }
    }

    printf("Highest mark among %d students = %d", n, largest);

    return 0;
}
```

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int marks[50];</code>	The size in the brackets must be a fixed number, so the array is made large enough for any class that is expected.
<code>scanf("%d", &n);</code>	Reads how many marks will follow. Everything after this line depends on n.
<code>for (i = 0; i < n; i++)</code>	Fills only the first n boxes. Boxes n to 49 are never touched, and that is perfectly fine.
<code>for (i = 1; i < n; i++)</code>	The search now stops at n too. Using 6 here by mistake would read boxes that were never filled.

OUTPUT 1 — FOUR STUDENTS

```
How many students are there? 4
Enter 4 marks: 62 91 45 78
Highest mark among 4 students = 91
```

OUTPUT 2 — SIX STUDENTS, DIFFERENT MARKS

```
How many students are there? 6
Enter 6 marks: 55 40 72 68 90 61
Highest mark among 6 students = 90
```

Q6. Admin Office — Sorting the Marks in Ascending Order

REAL SITUATION

The Admin office has to put the six answer sheets in order, from the lowest mark to the highest: 62, 91, 45, 78, 88, 53.

The clerk compares **two neighbouring sheets** at a time. If the left one has more marks than the right one, he swaps them. He goes along the row again and again until nothing needs swapping.

After the first walk the largest sheet has reached the far end, after the second walk the next largest is in place, and so on.

CONCEPT USED

Nested loops with a swap. The outer loop counts the walks, the inner loop does the comparing.

A swap needs a third variable, temp, to hold one value while the two are exchanged.

```
#include <stdio.h>

int main() {
    int marks[6] = {62, 91, 45, 78, 88, 53};
    int i, j, temp;

    printf("Before sorting: ");
    for (i = 0; i < 6; i++) {
        printf("%d ", marks[i]);
    }

    for (i = 0; i < 5; i++) {           // OUTER: one walk along the row
        for (j = 0; j < 5 - i; j++) {   // INNER: compare two neighbours
            if (marks[j] > marks[j + 1]) {
                temp = marks[j];        // the swap needs a spare box
                marks[j] = marks[j + 1];
                marks[j + 1] = temp;
            }
        }
    }

    printf("\nAfter sorting: ");
    for (i = 0; i < 6; i++) {
        printf("%d ", marks[i]);
    }

    return 0;
}
```

Step by step — the row after each walk

The outer loop makes five walks along the row. After every walk, the biggest value that was still loose has been pushed to the end, and the green boxes are the ones already in their final place.

62 [0]	91 [1]	45 [2]	78 [3]	88 [4]	53 [5]
-----------	-----------	-----------	-----------	-----------	-----------

Before sorting — the marks as they were entered

62 [0]	45 [1]	78 [2]	88 [3]	53 [4]	91 [5]
-----------	-----------	-----------	-----------	-----------	-----------

After walk 1 ($i = 0$): 91 has been pushed to the last box

45 [0]	62 [1]	78 [2]	53 [3]	88 [4]	91 [5]
-----------	-----------	-----------	-----------	-----------	-----------

After walk 2 ($i = 1$): 88 is now in place as well

45 [0]	62 [1]	53 [2]	78 [3]	88 [4]	91 [5]
-----------	-----------	-----------	-----------	-----------	-----------

After walk 3 ($i = 2$): 78 is in place

45 [0]	53 [1]	62 [2]	78 [3]	88 [4]	91 [5]
-----------	-----------	-----------	-----------	-----------	-----------

After walk 4 ($i = 3$): 62 is in place, and the whole row happens to be in order already

45 [0]	53 [1]	62 [2]	78 [3]	88 [4]	91 [5]
-----------	-----------	-----------	-----------	-----------	-----------

After walk 5 ($i = 4$): nothing left to swap — this is the printed result

Why the inner loop gets shorter every time

After walk 1 the last box is already correct, so walk 2 does not need to look at it. That is exactly what $j < 5 - i$ does: with $i = 0$ it compares up to box 5, with $i = 1$ only up to box 4, and so on. Comparing the finished part again would not be wrong, only wasted work.

AT THE ADMIN OFFICE	→	IN THE CODE
One full walk along the row of sheets	→	outer for ($i = 0; i < 5; i++$)
Comparing two neighbouring sheets	→	if ($\text{marks}[j] > \text{marks}[j + 1]$)
Exchanging the two sheets	→	temp = $\text{marks}[j]$; ... ; $\text{marks}[j+1] = \text{temp}$;
Not looking at the finished end again	→	$j < 5 - i$

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
outer for (i < 5)	Six sheets need at most five walks. After the last walk everything is already in place.
inner for (j < 5 - i)	Compares marks[j] with its neighbour marks[j + 1]. The range shrinks by one after every walk, because the end of the row is already sorted.
if (marks[j] > marks[j + 1])	The left sheet has more marks than the right one, so they are in the wrong order. Turning > into < sorts from highest to lowest instead.
temp = ...; ...; ... = temp;	The three lines of a swap. Without temp the first value would be lost the moment the second one is written over it.
j + 1 never passes the end	The inner condition is j < 5 - i, so the largest j is 4 and j + 1 is 5 — the last real box.
int marks[6] = { ... };	The six marks, filled at the start. Sorting changes this same array; no second array is used.
int i, j, temp;	Three helpers: i counts the walks, j moves along the row, and temp holds a value during a swap.
the "Before sorting" loop	Prints the row before any swapping, so the change can be seen. It must come before the sorting loops.
the "After sorting" loop	The same three lines again, printed after the sorting. Comparing the two output lines is the proof that the work happened.

OUTPUT

```
Before sorting: 62 91 45 78 88 53
After sorting:  45 53 62 78 88 91
```

Q7. IITA Pavilion — Reversing the Seat Order

REAL SITUATION

Six numbered chairs stand in a row at the Pavilion: 11, 12, 13, 14, 15, 16.

The order has to be turned around **for real** this time — not just read backwards. So the first chair is swapped with the last, the second with the second-last, and the third with the third-last.

After three swaps the whole row is reversed. Doing six swaps would bring every chair back to where it started, so the loop must stop at the middle.

CONCEPT USED

Swapping the two ends of an array.

Box i is swapped with box $5 - i$, and the loop runs only **half** the length of the array.

Compare with Question 3: there the array was untouched; here the array itself changes.

```
#include <stdio.h>

int main() {
    int seat[6] = {11, 12, 13, 14, 15, 16};
    int i, temp;

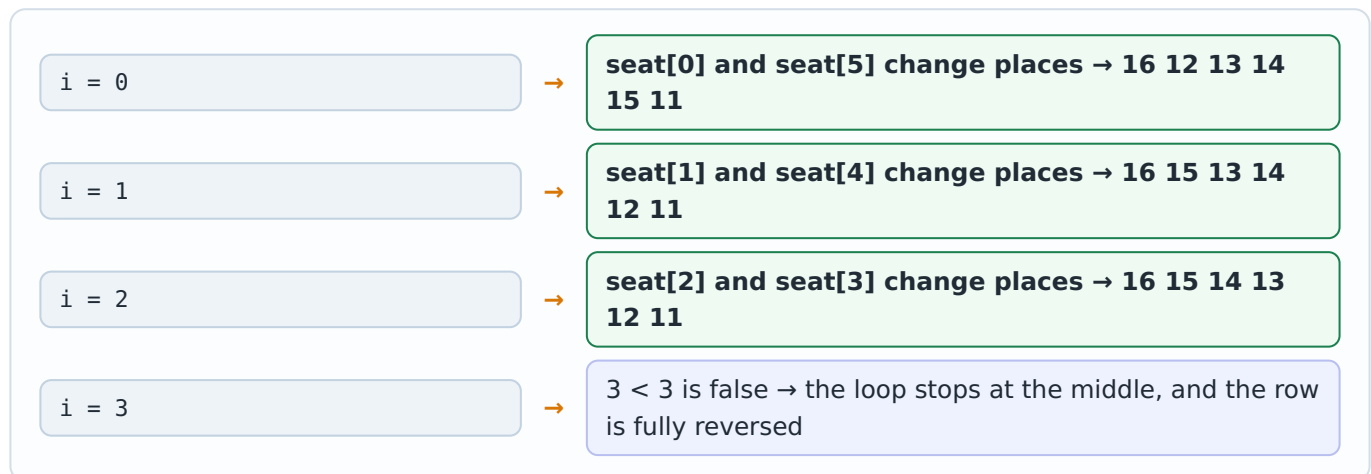
    printf("Original order: ");
    for (i = 0; i < 6; i++) {
        printf("%d ", seat[i]);
    }

    for (i = 0; i < 3; i++) {           // only half the row
        temp = seat[i];
        seat[i] = seat[5 - i];        // box i and box 5-i change places
        seat[5 - i] = temp;
    }

    printf("\nReversed order: ");
    for (i = 0; i < 6; i++) {
        printf("%d ", seat[i]);
    }

    return 0;
}
```

The three swaps



AT THE PAVILION	→	IN THE CODE
The first chair and the last chair	→	<code>seat[i] and seat[5 - i]</code>
Exchanging those two chairs	→	<code>temp = seat[i]; ... ; seat[5-i] = temp;</code>
Stopping once the middle is reached	→	<code>for (i = 0; i < 3; i++)</code>

The three swaps, seen as boxes



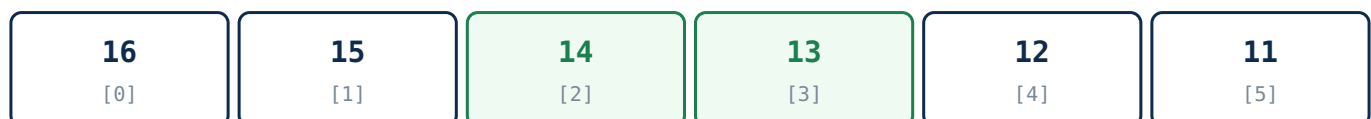
Before: the chairs in their original order



After swap 1 — box 0 and box 5 have changed places



After swap 2 — box 1 and box 4 have changed places



After swap 3 — box 2 and box 3 have changed places, and the row is fully reversed

Why the loop must stop at the middle

Three swaps are enough for six chairs, because every swap puts **two** chairs in place. If the loop ran to `i < 6`, swap 4 would exchange boxes 3 and 2 again, swap 5 would undo swap 2, and swap 6 would undo swap 1 — the row would come back exactly as it started.

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>for (i = 0; i < 3; i++)</code>	Half of 6 is 3. Running the loop to 6 would swap every pair twice and bring the row back to its original order.
<code>seat[5 - i]</code>	The partner box, counted from the far end: 5, then 4, then 3. For an array of n boxes it is $n - 1 - i$.
<code>temp</code>	Holds one chair while the two are exchanged, exactly as in Question 6.
<code>int seat[6] = { ... };</code>	The six chair numbers. This array is changed by the swaps, unlike Question 3 where the array stayed as it was.
<code>int i, temp;</code>	<code>i</code> counts the swaps, <code>temp</code> holds one chair while the two are exchanged.
the "Original order" loop	Prints the row before the swapping, so that the reversal is visible.
the "Reversed order" loop	Prints the same array after the swaps. The array itself has changed, so the same printing loop now shows a different order.

OUTPUT

```
Original order: 11 12 13 14 15 16
Reversed order: 16 15 14 13 12 11
```

Q8. Auditorium Register — Reading a Name and Counting the Letters

REAL SITUATION

At the Auditorium register you write your name in one go.

The computer stores it as a **row of letters**, one letter per box, and puts the end mark '\0' in the box just after the last letter.

To count the letters the program walks along the boxes and stops the moment it meets that end mark. It does not need to be told the length beforehand.

CONCEPT USED

Character array, the %s format, and the '\0' end mark.

`scanf("%s", name);` takes **no** &, because the array name already is the address of the first box.

Remember that %s stops at the first space, so only one word is read.

```
#include <stdio.h>

int main() {
    char name[20];           // room for 19 letters and the end mark
    int i = 0;

    printf("Enter your name (one word): ");
    scanf("%s", name);       // no & with %s

    printf("Name stored in the array: %s\n", name);

    while (name[i] != '\0') { // walk until the end mark
        printf("name[%d] = %c\n", i, name[i]);
        i++;
    }

    printf("Letters counted before '\\0' = %d", i);

    return 0;
}
```

S [0]	u [1]	b [2]	r [3]	a [4]	t [5]	a [6]	\0 [7] stop here
----------	----------	----------	----------	----------	----------	----------	---------------------

The loop prints boxes 0 to 6 and stops at box 7, so the count comes out as 7

AT THE AUDITORIUM REGISTER	→	IN THE CODE
Writing your whole name at once	→	<code>scanf("%s", name);</code>
The letters sitting in the boxes	→	<code>name[0], name[1], name[2] ...</code>
The end mark just after the last letter	→	<code>'\0'</code>
Counting the letters up to that mark	→	<code>while (name[i] != '\0')</code>

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>char name[20];</code>	Twenty boxes, each holding one character. The longest name that fits is 19 letters, because one box is needed for <code>'\0'</code> .
<code>scanf("%s", name);</code>	Reads one word and stores it, then adds the end mark itself. No <code>&</code> is written here.
<code>while (name[i] != '\0')</code>	Walks box by box. The end mark is what stops the loop, so the program never needs to know the length in advance.
<code>%c</code> and <code>%s</code>	<code>%c</code> prints one character out of one box; <code>%s</code> prints the whole row up to the end mark.
<code>"'\0'"</code> inside the message	A backslash has a special meaning inside a string, so it is written twice to print one backslash on the screen.
<code>int i = 0;</code>	The counter, started at 0 because the first letter sits in box 0. At the end it also serves as the letter count.
<code>printf("Enter your name ...");</code>	The prompt, printed once before the reading.
<code>printf("... %s\n", name);</code>	Prints the whole word in one go, to show that the array really holds the name.
<code>printf("name[%d] = %c\n", ...);</code>	Prints one box per line, so the letters can be seen sitting in their own boxes.

OUTPUT (THE NAME SUBRATA IS TYPED)

```
Enter your name (one word): Subrata
Name stored in the array: Subrata
name[0] = S
name[1] = u
name[2] = b
name[3] = r
name[4] = a
name[5] = t
name[6] = a
Letters counted before '\0' = 7
```

Q9. Extracting the Vowels from a Word

REAL SITUATION

A word is typed at the counter — for example the city name Allahabad.

The program has to pick out only the **vowels** — a, e, i, o, u — and count them, leaving the other letters alone.

It walks along the row of letters exactly as in Question 8, but at every box it now asks one extra question: *is this letter a vowel?*

CONCEPT USED

Walking a string to the end mark, with a test at each box.

The test uses `||` (logical OR) from Assignment 2, once for the small letters and once for the capitals, because 'A' and 'a' are different characters to C.

```
#include <stdio.h>

int main() {
    char word[20];
    int i = 0;
    int count = 0;
    char ch;

    printf("Enter a word: ");
    scanf("%s", word);

    printf("Vowels in the word: ");

    while (word[i] != '\0') {
        ch = word[i];
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
            printf("%c ", ch);
            count++;
        }
        i++;
    }

    printf("\nTotal vowels = %d", count);

    return 0;
}
```

The word in its boxes, with the vowels marked

A	l	l	a	h	a	b	a	d	\0
[0] vowel	[1]	[2]	[3] vowel	[4]	[5] vowel	[6]	[7] vowel	[8]	[9] stop

The loop walks from box 0 to box 8 and stops at the end mark in box 9. Four boxes hold a vowel, so count finishes at 4.

Step by step for the word Allahabad

word[0] = A	→	a vowel → printed, count becomes 1
word[1] = l	→	not a vowel → skipped
word[2] = l	→	not a vowel → skipped
word[3] = a	→	a vowel → printed, count becomes 2
word[4] ... word[8]	→	h, a, b, a, d → the two a's are printed, count reaches 4
word[9] = '\0'	→	the end mark → the loop stops, four vowels in all

WHILE READING THE WORD	→	IN THE CODE
Looking at one letter	→	ch = word[i];
"Is this letter a vowel?"	→	if (ch == 'a' ch == 'e' ...)
Writing it down and counting it	→	printf("%c ", ch); count++;
Moving on to the next letter	→	i++

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
char ch;	Holds the current letter, so the long condition does not have to repeat word[i] ten times.
the ten comparisons	Both cases are tested, because 'A' and 'a' are two different characters. Leaving out the capitals would miss the A of Allahabad.
count++	Counts only the vowels, so it sits inside the if. i++ counts every letter, so it sits outside .
i++ at the end of the body	Moves to the next box. If it were forgotten, the same letter would be tested for ever.
char word[20];	Twenty boxes for the word, one letter each, with the last box kept for the end mark.
int i = 0; int count = 0;	i walks along the letters; count counts only the vowels. They are two different jobs, so they are two different variables.
printf("Enter a word: "); scanf("%s", word);	The prompt and the reading. %s takes no &, and it stops at the first space.
printf("Vowels in the word:");	A label printed once before the loop, so the vowels appear after it on the same line.
while (word[i] != '\0')	Walks to the end mark. The length is never counted in advance; the end mark is what stops the loop.
printf("%c ", ch);	Prints one vowel with a space after it, so they line up on one line.

OUTPUT (THE WORD ALLAHABAD IS TYPED)

```
Enter a word: Allahabad  
Vowels in the word: A a a a  
Total vowels = 4
```

Q10. Pattern Matching — Searching for a Word Inside a Text

REAL SITUATION

Looking for a short word inside a longer one. The text is IIITALLAHABAD and the pattern being searched for is ALLAH.

You place the pattern under the text starting at the first letter and compare letter by letter. At the first mismatch you slide the pattern **one place to the right** and start comparing again.

If every letter of the pattern matches, the search is over and the starting position is reported. Here it matches from position 4.

CONCEPT USED

Nested loops over two character arrays.

The outer loop chooses the starting position *i* in the text; the inner loop compares `pattern[j]` with `text[i + j]`.

If the inner loop reaches the end mark of the pattern, every letter matched.

```
#include <stdio.h>

int main() {
    char text[30] = "IIITALLAHABAD";
    char pattern[10] = "ALLAH";
    int i, j;
    int found = 0;

    for (i = 0; text[i] != '\0'; i++) {           // try every starting position
        j = 0;
        while (pattern[j] != '\0' && text[i + j] == pattern[j]) {
            j++;                                  // letters matched so far
        }
        if (pattern[j] == '\0') {                 // the whole pattern matched
            printf("Pattern found at position %d\n", i);
            found = 1;
            break;
        }
    }

    if (found == 0) {
        printf("Pattern not found\n");
    }

    printf("Text      : %s\nPattern : %s", text, pattern);

    return 0;
}
```

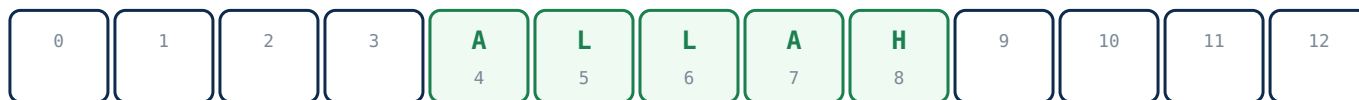
The pattern sliding under the text

I	I	I	T	A	L	L	A	H	A	B	A	D
0	1	2	3	4	5	6	7	8	9	10	11	12

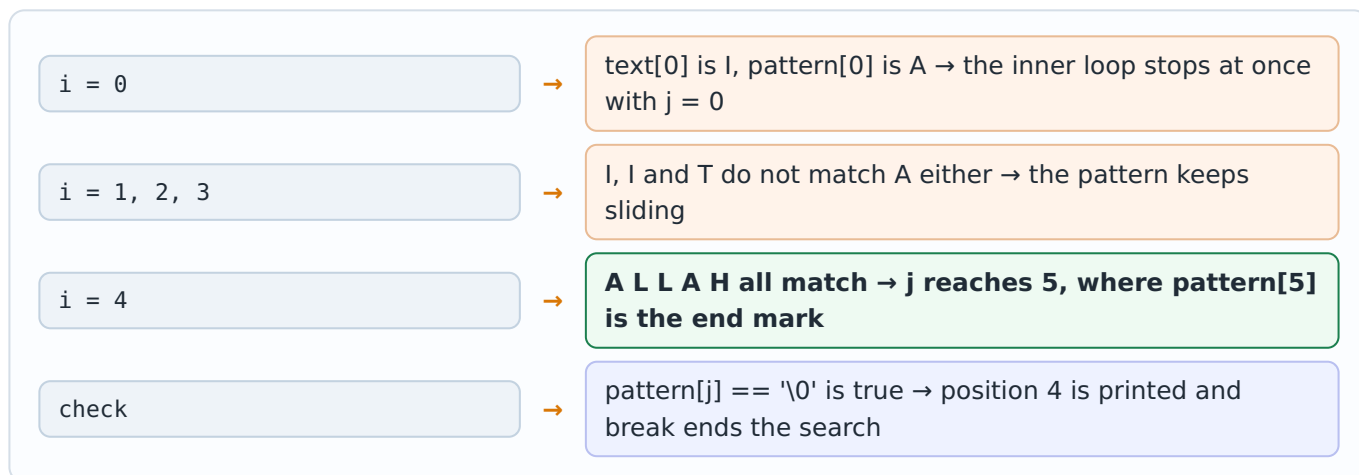
The text, with the index written under every letter



Attempt at position 0: the first letters are I and A — they do not match, so the pattern slides one place to the right



Attempt at position 4: A, L, L, A, H all match the letters above them, so the search succeeds and break ends it



WHILE SEARCHING THE TEXT	→	IN THE CODE
Placing the small word under the text	→	i, the starting position
Comparing letter by letter	→	text[i + j] == pattern[j]
Sliding one place to the right	→	i++
Every letter matched, so stop	→	pattern[j] == '\0' then break;

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>for (i = 0; text[i] != '\0'; i++)</code>	Slides the pattern along the text. The end mark of the text stops the search, so the length never has to be counted.
<code>text[i + j] == pattern[j]</code>	Compares the letter of the pattern with the letter standing under it. <i>i</i> is where the pattern is placed, <i>j</i> is how far the comparison has gone.
<code>pattern[j] != '\0' && ...</code>	The end mark is tested first , so that the comparison stops as soon as the pattern is finished.
<code>if (pattern[j] == '\0')</code>	The inner loop can stop for two reasons: a mismatch, or the pattern being finished. Only the second one means a match, and this test tells them apart.
<code>found and break</code>	<code>break</code> stops at the first match. <code>found</code> remembers whether anything was matched, so the "not found" message is printed only when it should be.
<code>char text[30] = "IIITALLAHABAD";</code>	The text being searched. C adds the end mark after the last letter, and the outer loop uses that mark to know where the text ends.
<code>char pattern[10] = "ALLAH";</code>	The word being looked for. Changing this one line changes the whole search.
<code>int i, j;</code>	<i>i</i> is where the pattern is placed on the text; <i>j</i> is how far the letter-by-letter comparison has gone.
<code>int found = 0;</code>	A yes-or-no marker, 0 meaning "nothing matched yet". It is needed because the loop can end in two different ways.
<code>j = 0;</code>	Reset at every new starting position, so each attempt compares the pattern from its first letter.
<code>if (found == 0) { ... }</code>	Runs only when the loop finished without a match, so the "not found" message never appears together with a "found" message.
<code>the last printf</code>	Prints the text and the pattern at the end, so the reader can check the reported position by counting the letters.

OUTPUT

```
Pattern found at position 4
Text    : IIITALLAHABAD
Pattern : ALLAH
```

Quick Revision — Concept, Real IIITA Memory, Code Shape

CONCEPT	REMEMBER IT AS	CODE SHAPE
What an array is	the hostel letter boxes: one name, many numbered boxes, all holding the same kind of thing	<code>int marks[5];</code>
Declaration	telling the carpenter the type and how many boxes	<code>type name[size];</code>
Memory	the boxes sit side by side, each the same size, so any box is reached at once	<code>sizeof(marks) = 5 × 4 = 20</code>
Index	the distance from the start, so the first box is 0 and the last is n - 1	<code>marks[0] ... marks[n-1]</code>
Initialisation	the clerk filling the boxes before the day starts; missing ones become 0	<code>int a[5] = {1, 2};</code>
Address operator	the postman needs the box, not the letter	<code>scanf("%d", &roll[i]);</code>
Reading with a loop	filling the boxes one at a time	<code>for (...) scanf("%d", &a[i]);</code>
Printing in reverse	walking back along the same board; nothing inside moves	<code>for (i = n-1; i >= 0; i--)</code>
Copying	moving the books one by one to the second shelf	<code>b[i] = a[i];</code>
Character array	a name is a row of letters, with '\0' marking the end	<code>char name[20];</code>
Reading a string	writing the whole name in the register at once, no & needed	<code>scanf("%s", name);</code>
Largest value	keeping the best answer sheet in your hand	<code>if (a[i] > largest) largest = a[i];</code>
Sorting	swapping neighbouring sheets until the row is in order	<code>temp = a[j]; a[j] = a[j+1]; a[j+1] = temp;</code>
Reversing the array	swapping the two ends and stopping at the middle	<code>for (i = 0; i < n/2; i++)</code>
Pattern matching	sliding the small word along the text until every letter matches	<code>text[i + j] == pattern[j]</code>

— End of Assignment 4 —